NotiUAM

Bitácora de desarrollo

Proyecto Terminal II

Aaron Rodrigo Ramos Reyes

Universidad Autónoma Metropolitana — Unidad Cuajimalpa
Licenciatura en Tecnologías y Sistemas de Información

2026

Índice general

1. Introducción **6**
- 1.1. Objetivo . . . 6
- 1.2. Organización . . . 6
- 1.3. Sesiones documentadas . . . 6
- 1.4. Convenciones . . . 7

2. Sesión 1: Infraestructura base con Docker **8**
- 2.1. Objetivo de la sesión . . . 8
- 2.2. Contexto . . . 8
- 2.3. Desarrollo . . . 8
  - 2.3.1. Estructura del repositorio . . . 8
  - 2.3.2. Variables de entorno (`.env`) . . . 9
  - 2.3.3. `docker-compose.yml` . . . 9
  - 2.3.4. Decisión técnica: imagen de Kafka . . . 11
  - 2.3.5. Modo KRaft en lugar de Zookeeper . . . 11
- 2.4. Verificación . . . 11
  - 2.4.1. Levantar los servicios . . . 11
  - 2.4.2. PostgreSQL . . . 12
  - 2.4.3. Redis . . . 12
  - 2.4.4. Kafka . . . 12
  - 2.4.5. Kafka UI . . . 12
- 2.5. Resultado . . . 12

3. Sesión 2: Backend Spring Boot base **14**
- 3.1. Objetivo de la sesión . . . 14
- 3.2. Contexto . . . 14
- 3.3. Desarrollo . . . 14
  - 3.3.1. Generación del proyecto . . . 14
  - 3.3.2. Dependencias adicionales . . . 15
  - 3.3.3. Entidades JPA . . . 15
  - 3.3.4. Repositorios Spring Data . . . 16
  - 3.3.5. Migración Flyway . . . 17
  - 3.3.6. Configuración (`application.yml`) . . . 18
  - 3.3.7. Clase principal . . . 19
- 3.4. Verificación . . . 19
- 3.5. Resultado . . . 20

4. Sesión 3: Seguridad JWT y autenticación **21**
4.1. Objetivo de la sesión . . . 21
4.2. Contexto . . . 21
4.3. Desarrollo . . . 21
4.3.1. Estructura creada . . . 21
4.3.2. Decisiones técnicas . . . 22
4.3.3. Componentes clave . . . 22
4.3.4. `AuthController` . . . 23
4.3.5. Manejo global de errores . . . 23
4.3.6. Ajustes al `application.yml` . . . 24
4.4. Verificación . . . 24
4.4.1. Registro . . . 24
4.4.2. Login . . . 24
4.4.3. Refresh . . . 24
4.4.4. Verificar hash en PostgreSQL . . . 25
4.4.5. Comprobar que arranca sin contraseña temporal . . . 25
4.5. Resultado . . . 25

5. Sesión 4: Canales, suscripciones y publicaciones **26**
5.1. Objetivo de la sesión . . . 26
5.2. Contexto . . . 26
5.3. Desarrollo . . . 26
5.3.1. Estructura creada . . . 26
5.3.2. Decisiones técnicas . . . 27
5.3.3. Componentes clave . . . 27
5.3.4. Cambios a archivos previos . . . 29
5.4. Verificación . . . 29
5.4.1. Preparar variables . . . 29
5.4.2. Crear, suscribirse y publicar . . . 29
5.4.3. Listar . . . 30
5.4.4. Verificar seguridad . . . 30
5.5. Resultado . . . 31

6. Sesión 5: Kafka productor, consumidor y notificaciones **32**
6.1. Objetivo de la sesión . . . 32
6.2. Contexto . . . 32
6.3. Desarrollo . . . 32
6.3.1. Estructura creada . . . 32
6.3.2. Decisiones técnicas . . . 33
6.3.3. Componentes clave . . . 33
6.3.4. Cambios a archivos previos . . . 35
6.3.5. Problemas encontrados y solución . . . 36
6.4. Verificación . . . 36
6.4.1. Preparar dos usuarios . . . 36
6.4.2. Publicar y observar los logs . . . 36
6.4.3. Consultar y limpiar contadores . . . 37
6.4.4. Verificación en Redis . . . 37
6.4.5. Verificación en Kafka UI . . . 38

6.5. Resultado . . . 38

7. Sesión 6a: Setup del cliente Flutter y autenticación 39
7.1. Objetivo de la sesión . . . 39
7.2. Contexto . . . 39
7.3. Desarrollo . . . 39
7.3.1. Generación del proyecto . . . 39
7.3.2. Dependencias . . . 40
7.3.3. Estructura de carpetas . . . 40
7.3.4. Decisiones técnicas . . . 40
7.3.5. Detección de plataforma para la URL del backend . . . 41
7.3.6. Configuración Android . . . 41
7.3.7. Cambios a archivos previos . . . 41
7.3.8. Problemas encontrados y solución . . . 41
7.4. Verificación . . . 42
7.4.1. Arranque . . . 42
7.4.2. Flujo verificado . . . 43
7.5. Resultado . . . 43

8. Sesión 6b: Listado, búsqueda y creación de canales 44
8.1. Objetivo de la sesión . . . 44
8.2. Contexto . . . 44
8.3. Desarrollo . . . 44
8.3.1. Archivos nuevos . . . 44
8.3.2. Archivos modificados . . . 45
8.3.3. Decisiones técnicas . . . 45
8.3.4. Componentes clave . . . 45
8.3.5. Cambios a archivos previos . . . 47
8.4. Verificación . . . 47
8.4.1. Compilación . . . 47
8.4.2. Flujo verificado . . . 47
8.5. Resultado . . . 47

9. Sesión 6c: Publicaciones y contadores de no leídos en Flutter 49
9.1. Objetivo de la sesión . . . 49
9.2. Contexto . . . 49
9.3. Desarrollo . . . 49
9.3.1. Archivos nuevos . . . 49
9.3.2. Archivos modificados . . . 49
9.3.3. Decisiones tecnicas . . . 50
9.3.4. Componentes clave . . . 50
9.3.5. Cambios a archivos previos . . . 51
9.3.6. Problemas encontrados y solucion . . . 51
9.4. Verificacion . . . 51
9.5. Resultado . . . 52

10. Rediseno UX/UI con paleta UAM-C **53**
10.1. Objetivo . . . 53
10.2. Contexto . . . 53
10.3. Desarrollo . . . 53
10.3.1. Diagnostico del problema de iconos . . . 53
10.3.2. Correccion del `pubspec.yaml` . . . 53
10.3.3. Paleta institucional . . . 54
10.3.4. Centralizacion del tema . . . 54
10.3.5. Rediseno de pantallas . . . 55
10.3.6. Cambios a archivos previos . . . 55
10.4. Verificacion . . . 55
10.5. Resultado . . . 56

11. Backend: reportes, votacion y roles **57**
11.1. Objetivo . . . 57
11.2. Contexto . . . 57
11.3. Desarrollo . . . 57
11.3.1. Migracion V2 . . . 57
11.3.2. Enumeraciones . . . 58
11.3.3. Entidades JPA . . . 58
11.3.4. Repositorios . . . 59
11.3.5. `ReportService` . . . 59
11.3.6. `AdminUserService` . . . 60
11.3.7. Moderacion directa por dueno del canal . . . 61
11.3.8. Endpoints . . . 62
11.3.9. Habilitacion de `@PreAuthorize` . . . 62
11.3.10. Problemas encontrados y solucion . . . 62
11.4. Verificacion . . . 62
11.4.1. Reglas de rol . . . 62
11.4.2. Flujo de reporte y votacion . . . 63
11.4.3. Moderacion directa . . . 63
11.5. Resultado . . . 63

12. Flutter: reportar, moderar y administrar **64**
12.1. Objetivo . . . 64
12.2. Contexto . . . 64
12.3. Desarrollo . . . 64
12.3.1. Modelos nuevos . . . 64
12.3.2. Providers nuevos . . . 64
12.3.3. Wigets reutilizables . . . 65
12.3.4. Menu contextual en publicaciones . . . 65
12.3.5. Pantalla `AdminScreen` . . . 65
12.3.6. `AdminReportsTab` . . . 66
12.3.7. `ReportDetailScreen` . . . 66
12.3.8. `AdminUsersTab` . . . 66
12.3.9. Bottom sheet de acciones . . . 67
12.3.10. `AdminAuditTab` . . . 67
12.3.11. Cambios a archivos previos . . . 67

12.4. Verificacion 67
12.5. Resultado 67

13. Distintivo visual de canales institucionales **69**
13.1. Objetivo 69
13.2. Contexto 69
13.3. Desarrollo 69
13.3.1. Cambio en el backend 69
13.3.2. Modelo en Flutter 70
13.3.3. Wiget `ChannelOwnerBadge` 70
13.3.4. Ubicaciones del badge 70
13.4. Verificacion 71
13.5. Resultado 71

Capítulo 1

Introducción

Este documento es la **bitácora técnica** del desarrollo de **NotiUAM**, la plataforma distribuida de comunicación y alertas comunitarias para entornos universitarios descrita en el protocolo de Proyecto Terminal I.

1.1. Objetivo

Registrar de forma cronológica y reproducible cada una de las sesiones de desarrollo: qué se hizo, por qué se hizo así, qué archivos se crearon, qué comandos se ejecutaron y cómo se verificó que todo funcionara.

1.2. Organización

Cada capítulo corresponde a una **sesión de trabajo** y sigue la misma estructura:

1. **Objetivo de la sesión.** Qué se busca lograr.
2. **Contexto.** Qué teníamos antes y qué necesitamos ahora.
3. **Desarrollo.** Archivos, comandos y decisiones técnicas.
4. **Verificación.** Cómo comprobar que funcionó.
5. **Resultado.** Estado al cierre de la sesión y pendientes.

1.3. Sesiones documentadas

- **Sesión 1:** Infraestructura base con Docker.
- **Sesión 2:** Backend Spring Boot base con entidades JPA, repositorios y Flyway.
- **Sesión 3:** Seguridad JWT con registro, login y refresh.
- **Sesión 4:** Endpoints de canales, suscripciones y publicaciones.
- **Sesión 5:** Productor y consumidor Kafka, contadores en Redis.
- **Sesión 6a:** Cliente Flutter: setup, HTTP con JWT, login y registro.

- **Sesión 6b:** Listado, búsqueda, detalle y creación de canales en Flutter.
- **Sesión 6c:** Publicaciones y contadores de no leídos en Flutter.
- **Rediseño UX/UI:** Paleta UAM-C y corrección de iconos de Material.
- **Backend de reportes y roles:** Votación colegiada, gestión de roles y auditoría.
- **Interfaz de administración:** Reportar, moderar, votar, gestionar usuarios y consultar auditoría.
- **Distintivo de canales institucionales:** Badges visuales según rol del creador.

1.4. Convenciones

- Los bloques con fondo gris son **código o comandos** listos para copiar.
- Las cajas azules con **Nota** contienen aclaraciones técnicas.
- Las cajas amarillas con **Aviso** marcan advertencias o cosas a tener cuidado.
- Las cajas verdes con **Verificación** indican cómo comprobar que algo funciona.

Capítulo 2

Sesión 1: Infraestructura base con Docker

2.1. Objetivo de la sesión

Levantar un entorno de desarrollo reproducible con todos los servicios de infraestructura que NotiUAM necesita, orquestados con Docker Compose:

- **PostgreSQL 16** para persistencia relacional.
- **Redis 7** para caché y datos temporales.
- **Apache Kafka 3.7** (modo KRaft, sin Zookeeper) como broker de eventos.
- **Kafka UI** para inspeccionar topics y mensajes desde el navegador.

2.2. Contexto

El protocolo describe una arquitectura pub-sub con Kafka como broker central, PostgreSQL y Redis para persistencia y caché, y contenerización con Docker para reproducibilidad (secciones 2.3 y 6.2.3). Antes de escribir una sola línea de código Java o Dart, necesitábamos poder levantar esa infraestructura con un solo comando.

2.3. Desarrollo

2.3.1. Estructura del repositorio

Se creó el siguiente árbol en la raíz del proyecto:

```
notiuam/
|- docker-compose.yml
|- .env
|- .gitignore
|- README.md
|- backend
```

2.3.2. Variables de entorno (`.env`)

Se centralizaron credenciales y puertos en un archivo `.env` para no incrustarlos en el compose:

```
POSTGRES_DB=notiuam
POSTGRES_USER=notiuam
POSTGRES_PASSWORD=notiuam_dev_pass
POSTGRES_PORT=5432

REDIS_PORT=6379

KAFKA_PORT=9092
KAFKA_UI_PORT=8080

BACKEND_PORT=8081
JWT_SECRET=
    change_me_in_prod_please_use_a_very_long_random_secret_at_least_256_bits

JWT_EXPIRATION_MS=3600000
JWT_REFRESH_EXPIRATION_MS=604800000
```

Aviso

El `JWT_SECRET` de ejemplo **no debe usarse en producción**. Se reemplazará por un secreto de al menos 256 bits generado aleatoriamente.

2.3.3. `docker-compose.yml`

Se definieron cuatro servicios en una red interna `notiuam-net`, con volúmenes persistentes para cada uno.

```
networks:
  notiuam-net:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  kafka_data:

services:

  postgres:
    image: postgres:16-alpine
    container_name: notiuam-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "${POSTGRES_PORT}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
    networks: [notiuam-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${
          POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: notiuam-redis
    restart: unless-stopped
    command: ["redis-server", "--appendonly", "yes"]
    ports:
      - "${REDIS_PORT}:6379"
    volumes:
      - redis_data:/data
    networks: [notiuam-net]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  kafka:
    image: apache/kafka:3.7.0
    container_name: notiuam-kafka
    restart: unless-stopped
    ports:
      - "${KAFKA_PORT}:9092"
    environment:
      KAFKA_NODE_ID: "1"
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_LISTENERS: "PLAINTEXT://0.0.0.0:9092,CONTROLLER
          ://0.0.0.0:9093"
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://localhost:${KAFKA_PORT}"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: "CONTROLLER:PLAINTEXT,
          PLAINTEXT:PLAINTEXT"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: "0"
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
      KAFKA_NUM_PARTITIONS: "3"
    volumes:
      - kafka_data:/var/lib/kafka/data
    networks: [notiuam-net]
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-cluster.sh cluster-id --
          bootstrap-server localhost:9092 || exit 1"]
      interval: 15s
```

```
      timeout: 10s
      retries: 10

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    container_name: notiuam-kafka-ui
    restart: unless-stopped
    depends_on:
      kafka:
        condition: service_healthy
    ports:
      - "${KAFKA_UI_PORT}:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: notiuam-local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
      KAFKA_CLUSTERS_0_READONLY: "false"
    networks: [notiuam-net]
```

2.3.4. Decisión técnica: imagen de Kafka

Cambio respecto al plan original

Inicialmente se usó `bitnami/kafka:3.7`. Sin embargo, **Bitnami retiró su catálogo público de Docker Hub en septiembre de 2025**, por lo que el `docker pull` fallaba con `manifest unknown`. Se migró a la imagen oficial `apache/kafka:3.7.0`, mantenida por la Apache Software Foundation.
El cambio implicó ajustar los nombres de las variables de entorno:

- Bitnami usaba el prefijo `KAFKA_CFG_*` (p. ej. `KAFKA_CFG_NODE_ID`).
- Apache usa el prefijo `KAFKA_*` (p. ej. `KAFKA_NODE_ID`).

2.3.5. Modo KRaft en lugar de Zookeeper

Kafka 3.7 permite operar sin Zookeeper mediante **KRaft** (Kafka Raft). Se eligió este modo porque:

- Reduce el número de contenedores (uno en lugar de dos).
- Es el modo recomendado desde Kafka 3.0 y el único soportado a partir de Kafka 4.x.
- Simplifica el arranque en un solo nodo combinado *broker + controller*.

2.4. Verificación

2.4.1. Levantar los servicios

```
cd ~/Escritorio/notiuam
cp .env .env.example    # solo la primera vez, para tener un ejemplo
docker compose up -d
docker compose ps
```

Resultado esperado

Cuatro contenedores en estado `Up (healthy)`:

```
NAME                 STATUS
notiuam-postgres     Up (healthy)
notiuam-redis        Up (healthy)
notiuam-kafka        Up (healthy)
notiuam-kafka-ui     Up
```

2.4.2. PostgreSQL

```
docker exec -it notiuam-postgres psql -U notiuam -d notiuam -c "\l"
```

Debe listar la base `notiuam`.

2.4.3. Redis

```
docker exec -it notiuam-redis redis-cli ping
# Respuesta esperada: PONG
```

2.4.4. Kafka

```
# Crear topic de prueba
docker exec -it notiuam-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic test-topic --partitions 1 --replication-factor 1

# Listar topics
docker exec -it notiuam-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
# Debe aparecer: test-topic
```

2.4.5. Kafka UI

Abrir http://localhost:8080 en el navegador. Debe aparecer el cluster `notiuam-local` y el topic `test-topic` creado en el paso anterior.

2.5. Resultado

Estado al cierre

- Infraestructura base funcionando en Docker.
- Volúmenes persistentes para PostgreSQL, Redis y Kafka.
- Healthchecks configurados para detectar servicios listos.
- Archivo `.gitignore` y `README.md` en su lugar.

Pendiente para la siguiente sesión: crear el proyecto backend con Spring Boot y conectarlo a estos servicios.

Capítulo 3

Sesión 2: Backend Spring Boot base

3.1. Objetivo de la sesión

Crear el proyecto backend en Spring Boot 4.1.1 con Java 21, definir las entidades JPA del dominio (`User`, `Channel`, `Post`, `Subscription`), sus repositorios, la migración Flyway inicial y la conexión a PostgreSQL, Redis y Kafka.

3.2. Contexto

La infraestructura ya estaba arriba (Sesión 2). Faltaba el esqueleto del backend que consumiera esos servicios y validara el modelo relacional descrito en la Fase 2 de la metodología.

3.3. Desarrollo

3.3.1. Generación del proyecto

Se usó Spring Initializr vía `curl`. La primera tentativa con `bootVersion=3.3.5` falló porque esa versión ya no existe en el catálogo. El metadata disponible reportaba como versión por defecto `4.1.1.RELEASE`.

```
cd ~/Escritorio/notiuam
curl -s https://start.spring.io/starter.tgz \
  -d type=maven-project \
  -d language=java \
  -d bootVersion=4.1.1.RELEASE \
  -d baseDir=backend \
  -d groupId=mx.uam \
  -d artifactId=notiuam-backend \
  -d name=notiuam-backend \
  -d packageName=mx.uam.notiuam \
  -d javaVersion=21 \
  -d dependencies=web,data-jpa,data-redis,security,validation,kafka,
     postgresql,lombok,flyway \
  -o backend.tgz
tar -xzvf backend.tgz
rm backend.tgz
```

Nota sobre alias

En Spring Boot 4 el starter web se llama `spring-boot-starter-webmvc`, pero el **alias del Initializr sigue siendo `web`**. El alias `webmvc` no es válido y produce error `Invalid dependency`.

3.3.2. Dependencias adicionales

Se agregaron a `pom.xml` las librerías para manejo de JWT y para documentación OpenAPI:

```
<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>

<!-- OpenAPI / Swagger UI -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

3.3.3. Entidades JPA

Se modelaron cuatro entidades correspondientes a las tablas del protocolo: `users`, `channels`, `posts` y `subscriptions`.

`User`

```
@Entity
@Table(name = "users",
       uniqueConstraints = @UniqueConstraint(columnNames = "email"))
@EntityListeners(AuditingEntityListener.class)
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class User {

    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, length = 180)
```

```
    private String email;

    @Column(name = "password_hash", nullable = false, length = 100)
    private String passwordHash;

    @Column(name = "display_name", nullable = false, length = 120)
    private String displayName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Role role;

    @Column(nullable = false)
    @Builder.Default
    private boolean enabled = true;

    @CreatedDate @Column(name = "created_at", nullable = false,
        updatable = false)
    private Instant createdAt;

    @LastModifiedDate @Column(name = "updated_at")
    private Instant updatedAt;
}
```

Enumeraciones del dominio

```
public enum Role { STUDENT, PROFESSOR, ADMIN }

public enum ChannelCategory {
    ACADEMIC, CULTURAL, SPORTS, ADMINISTRATIVE, SOCIAL, OTHER
}
```

`Channel`, `Post` y `Subscription`

Las tres entidades siguen el mismo patrón: UUID como clave primaria, relaciones `@ManyToOne` hacia `User` y `Channel` cuando aplica, y campos de auditoría con `@CreatedDate`.

La entidad `Subscription` usa una **clave primaria compuesta** (`userId`, `channelId`) implementada con `@IdClass` y una clase auxiliar `SubscriptionId`.

3.3.4. Repositorios Spring Data

Cuatro interfaces que extienden `JpaRepository`:

```
public interface UserRepository extends JpaRepository<User, UUID> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

public interface ChannelRepository extends JpaRepository<Channel, UUID>
    {
    List<Channel> findByPublicChannelTrueOrderByCreatedAtDesc();
```

```
    List<Channel> findByCategoryAndPublicChannelTrue(ChannelCategory
        category);
    List<Channel> findByNameContainingIgnoreCaseAndPublicChannelTrue(
        String q);
    List<Channel> findByOwnerId(UUID ownerId);
}

public interface PostRepository extends JpaRepository<Post, UUID> {
    Page<Post> findByChannelIdOrderByCreatedAtDesc(UUID channelId,
        Pageable pageable);
}

public interface SubscriptionRepository
        extends JpaRepository<Subscription, SubscriptionId> {
    List<Subscription> findByUserId(UUID userId);
    List<Subscription> findByChannelId(UUID channelId);
    boolean existsByUserIdAndChannelId(UUID userId, UUID channelId);
    long countByChannelId(UUID channelId);
}
```

3.3.5. Migración Flyway

Se eligió Flyway en lugar de `ddl-auto=update` porque el protocolo pide trazabilidad del esquema (Fase 2). El archivo `V1__init.sql` crea las cuatro tablas con sus índices:

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           VARCHAR(180) NOT NULL UNIQUE,
    password_hash   VARCHAR(100) NOT NULL,
    display_name    VARCHAR(120) NOT NULL,
    role            VARCHAR(20)  NOT NULL,
    enabled         BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ
);

CREATE TABLE channels (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name         VARCHAR(120) NOT NULL,
    description  VARCHAR(500),
    category     VARCHAR(30)  NOT NULL,
    owner_id     UUID         NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
    is_public    BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_channels_public    ON channels(is_public);
CREATE INDEX idx_channels_category  ON channels(category);
CREATE INDEX idx_channels_owner     ON channels(owner_id);

CREATE TABLE posts (
```

```
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    channel_id  UUID          NOT NULL REFERENCES channels(id) ON DELETE
        CASCADE,
    author_id   UUID          NOT NULL REFERENCES users(id)    ON DELETE
        CASCADE,
    content     VARCHAR(2000) NOT NULL,
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_posts_channel_created ON posts(channel_id, created_at
   DESC);

CREATE TABLE subscriptions (
    user_id        UUID        NOT NULL REFERENCES users(id)    ON
       DELETE CASCADE,
    channel_id     UUID        NOT NULL REFERENCES channels(id) ON
       DELETE CASCADE,
    subscribed_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, channel_id)
);

CREATE INDEX idx_subs_channel ON subscriptions(channel_id);
```

3.3.6. Configuración (`application.yml`)

Se conectó el backend a los tres servicios de la Sesión 1:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/notiuam
    username: notiuam
    password: notiuam_dev_pass
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2

  jpa:
    hibernate:
      ddl-auto: validate          # Flyway maneja el esquema
    open-in-view: false

  flyway:
    enabled: true
    locations: classpath:db/migration

  data:
    redis:
      host: localhost
      port: 6379
      repositories:
        enabled: false            # Evita el warning "Could not safely
   identify store"

  kafka:
```

```
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.
    StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.
          JsonSerializer
    consumer:
      group-id: notiuam
      auto-offset-reset: earliest

server:
  port: 8081

jwt:
  secret: $JWT_SECRET:change_me_in_prod
  expiration-ms: 3600000
  refresh-expiration-ms: 604800000
```

3.3.7. Clase principal

```
@SpringBootApplication
@EnableJpaAuditing
public class NotiuamApplication {
    public static void main(String[] args) {
        SpringApplication.run(NotiuamApplication.class, args);
    }
}
```

3.4. Verificación

Arrancar la aplicación:

```
cd ~/Escritorio/notiuam/backend
./mvnw spring-boot:run
```

En la salida deben verse, en este orden:

1. `HikariPool-1 - Start completed` (conexión a PostgreSQL).
2. `Successfully applied 1 migration to schema "public"` (Flyway).
3. `Initialized JPA EntityManagerFactory` (Hibernate validó las entidades).
4. `Tomcat started on port 8081`.
5. `Started NotiuamApplication in X seconds`.

Verificar las tablas:

```
docker exec -it notiuam-postgres psql -U notiuam -d notiuam -c "\dt"
```

Resultado esperado

Las tablas `users`, `channels`, `posts`, `subscriptions` y `flyway_schema_history`.

3.5. Resultado

Estado al cierre

- Backend compila y arranca en el puerto 8081.
- Modelo relacional creado y validado por Hibernate.
- Conexión a PostgreSQL, Redis y Kafka configurada.
- Aún sin endpoints ni seguridad: Spring Security generaba una contraseña temporal de desarrollo.

Pendiente para la siguiente sesión: implementar autenticación JWT con endpoints de registro, login y refresh.

Capítulo 4

Sesión 3: Seguridad JWT y autenticación

4.1. Objetivo de la sesión

Implementar el sistema de autenticación basado en JSON Web Tokens (JWT), cubriendo:

- Registro de usuarios (`POST /api/auth/register`).
- Inicio de sesión (`POST /api/auth/login`).
- Renovación de token (`POST /api/auth/refresh`).
- Filtro que valida el JWT en cada petición autenticada.
- Configuración de Spring Security en modo *stateless*.
- Hash de contraseñas con BCrypt.

4.2. Contexto

En la Sesión 3 el backend arrancaba pero Spring Security generaba una contraseña temporal y todos los endpoints (salvo los públicos por defecto) quedaban bloqueados con `inMemoryUserDetailsManager`. No existía forma de crear usuarios en la base de datos ni de autenticarse contra ella.

4.3. Desarrollo

4.3.1. Estructura creada

```
src/main/java/mx/uam/notiuam/
|- config/
|   ‘- SecurityConfig.java
|- security/
|   |- JwtService.java
|   |- JwtAuthenticationFilter.java
|   ‘- UserDetailsServiceImpl.java
|- dto/auth/
|   |- RegisterRequest.java
|   |- LoginRequest.java
|   |- RefreshRequest.java
```

```
|   `- AuthResponse.java
|- service/
|   `- AuthService.java
|- controller/
|   `- AuthController.java
`- exception/
    `- GlobalExceptionHandler.java
```

4.3.2. Decisiones técnicas

- **JWT stateless.** No se usan sesiones HTTP. Toda la identidad viaja en el token firmado con HMAC-SHA256.
- **Dos tipos de token.** *Access token* de 1 hora y *refresh token* de 7 días. El claim `type` distingue uno del otro.
- **BCrypt.** Las contraseñas se almacenan siempre hasheadas, nunca en claro.
- **Alias `ROLE_`.** Los roles se exponen como `ROLE_STUDENT`, `ROLE_PROFESSOR`, `ROLE_ADMIN` para que `@PreAuthorize` funcione sin configuración extra.
- **CORS abierto en desarrollo.** Se permite cualquier origen (`*`) para que la app Flutter pueda probar sin configuración adicional. En producción se restringirá.

4.3.3. Componentes clave

`JwtService`

Genera, firma y valida tokens. Métodos principales:

```java
public String generateAccessToken(User user);
public String generateRefreshToken(User user);
public Claims parse(String token);
public boolean isValid(String token);
public long getAccessExpirationMs();
```

El claim `subject` guarda el `UUID` del usuario. Otros claims: `email`, `role`, `type` (`access` o `refresh`).

`JwtAuthenticationFilter`

Extiende `OncePerRequestFilter`. En cada petición:

1. Lee el header `Authorization: Bearer <token>`.
2. Valida el token con `JwtService`.
3. Extrae `userId`, `email`, `role`.
4. Coloca un `UsernamePasswordAuthenticationToken` en el `SecurityContext`.
5. El principal es un record `AuthenticatedUser(UUID id, String email, String role)`.

Nota

Si el token no es válido o falta, el filtro simplemente deja pasar la petición sin autenticación. La decisión de permitirla o rechazarla la toma la cadena de filtros de Spring Security según el matcher de la URL.

`SecurityConfig`

Define:

- `PasswordEncoder` con BCrypt.
- `AuthenticationManager` respaldado por `DaoAuthenticationProvider` + `UserDetailsServiceImpl`.
- `SecurityFilterChain`:
  - CSRF deshabilitado (API stateless).
  - Sesiones en `STATELESS`.
  - `/api/auth/**`, Swagger y `/actuator/health` públicos.
  - Todo lo demás, autenticado.
  - `JwtAuthenticationFilter` antes de `UsernamePasswordAuthenticationFilter`.

`AuthService`

Contiene la lógica de negocio:

- `register`: valida unicidad del email, hashea la contraseña, guarda y devuelve par de tokens.
- `login`: usa el `AuthenticationManager` para validar credenciales, luego emite tokens.
- `refresh`: valida el refresh token, verifica el claim `type=refresh` y emite un nuevo par.

4.3.4. `AuthController`

Expone los tres endpoints bajo `/api/auth`:

```
@PostMapping("/register")   // 201 Created
@PostMapping("/login")      // 200 OK
@PostMapping("/refresh")    // 200 OK
```

Todas las peticiones usan `@Valid` para disparar la validación de los DTOs.

4.3.5. Manejo global de errores

`GlobalExceptionHandler` con `@RestControllerAdvice` convierte:

- `IllegalArgumentException` → `400 Bad Request`.
- `MethodArgumentNotValidException` → `400` con detalle de campos inválidos.

Respuesta uniforme:

```
{
  "timestamp": "2026-10-05T22:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "El correo ya esta registrado"
}
```

4.3.6. Ajustes al `application.yml`

Se limpiaron dos warnings que aparecieron en la Sesión 3:

- Se eliminó `hibernate.dialect` (Hibernate 7 lo autodetecta).
- Se añadió `spring.data.redis.repositories.enabled: false` para evitar el escaneo de repositorios Redis.

4.4. Verificación

4.4.1. Registro

```
curl -s -X POST http://localhost:8081/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "aaron@uam.mx",
    "password": "password123",
    "displayName": "Aaron Ramos",
    "role": "STUDENT"
  }' | jq
```

Respuesta esperada

Un JSON con `accessToken`, `refreshToken`, `tokenType`, `expiresInMs` y un objeto `user` con los datos del usuario creado.

4.4.2. Login

```
curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aaron@uam.mx","password":"password123"}' | jq
```

Debe devolver un par de tokens nuevos.

4.4.3. Refresh

```
REFRESH=$(curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aaron@uam.mx","password":"password123"}' | jq -r .
     refreshToken)

```

```
curl -s -X POST http://localhost:8081/api/auth/refresh \
  -H "Content-Type: application/json" \
  -d "{\"refreshToken\":\"$REFRESH\"}" | jq
```

Debe emitir un nuevo par de tokens.

4.4.4. Verificar hash en PostgreSQL

```
docker exec -it notiuam-postgres psql -U notiuam -d notiuam \
  -c "SELECT email, display_name, role,
            LEFT(password_hash, 7) AS hash_prefix
     FROM users;"
```

Resultado esperado

El prefijo del hash debe ser `$2a$10$` (BCrypt). Nunca aparece la contraseña en claro.

4.4.5. Comprobar que arranca sin contraseña temporal

Al reiniciar el backend ya **no** debe aparecer el mensaje:

```
Using generated security password: ...
```

Si aparece, revisar que `UserDetailsServiceImpl` esté en el paquete `mx.uam.notiuam.security` y que la dependencia `spring-boot-starter-security` esté en el `pom.xml`.

4.5. Resultado

Estado al cierre

- Registro, login y refresh funcionales.
- Autenticación stateless con JWT (access 1h + refresh 7d).
- Contraseñas hasheadas con BCrypt.
- Roles expuestos como `ROLE_*` para usar `@PreAuthorize`.
- CORS listo para el cliente Flutter.
- Manejo global de errores unificado.

Pendiente para la siguiente sesión: endpoints de canales, suscripciones y publicaciones, usando el `AuthenticatedUser` del `SecurityContext` para identificar al emisor.

Capítulo 5

Sesión 4: Canales, suscripciones y publicaciones

5.1. Objetivo de la sesión

Implementar los endpoints REST del dominio principal de NotiUAM:

- CRUD de canales temáticos (`/api/channels`).
- Suscripción y desuscripción idempotentes (`/api/channels/{id}/subscribe`).
- Publicación y listado paginado de avisos (`/api/channels/{id}/posts`).
- Control de acceso basado en el usuario autenticado (dueño del canal, autor del post, rol admin).

5.2. Contexto

En la Sesión 4 quedó listo el sistema de autenticación JWT. Sin embargo, el backend solo exponía `/api/auth/**`; no existía ningún endpoint de negocio. Tampoco había una forma limpia de obtener el usuario autenticado desde los servicios, ni excepciones específicas del dominio para mapear a códigos HTTP (404, 403).

5.3. Desarrollo

5.3.1. Estructura creada

```
src/main/java/mx/uam/notiuam/
|- config/
|   `- CurrentUser.java                  (helper de usuario autenticado)
|- dto/channel/
|   |- ChannelCreateRequest.java
|   |- ChannelUpdateRequest.java
|   |- ChannelResponse.java
|   `- ChannelSummaryResponse.java
|- dto/post/
|   |- PostCreateRequest.java
|   `- PostResponse.java
```

```
|- service/
|   |- ChannelService.java
|   |- SubscriptionService.java
|   `- PostService.java
|- controller/
|   `- ChannelController.java
`- exception/
    |- NotFoundException.java
    `- ForbiddenException.java
```

5.3.2. Decisiones técnicas

- **Separación controller / service.** Los controllers no acceden directamente a los repositorios; toda la lógica vive en la capa de servicio.
- **UUID como identificador público.** A diferencia de un `Long autoincremental`, los UUID evitan que se pueda enumerar recursos (`/api/channels/1`, `/2`, ...).
- **Suscripción idempotente.** Llamar dos veces al endpoint de suscribirse no devuelve error; el segundo intento es un no-op. Esto simplifica el cliente móvil y evita problemas por reintentos de red.
- **Control de acceso en servicio, no en controller.** El `ChannelService` sabe quién es el dueño del canal y quién es admin; el controller solo delega.
- **Paginación en posts.** Los listados de publicaciones usan `Pageable` de Spring Data con un tope de 50 elementos por página para evitar respuestas gigantes.

5.3.3. Componentes clave

`CurrentUser`

Helper inyectable que lee el `SecurityContext` y devuelve el principal tipado. Antes, cada controller tendría que hacer un `cast` manual; ahora basta con inyectar y llamar a `currentUser.requireId()`.

```java
public AuthenticatedUser require() {
    Authentication auth = SecurityContextHolder.getContext().
        getAuthentication();
    if (auth == null || !(auth.getPrincipal() instanceof
        AuthenticatedUser u)) {
         throw new AccessDeniedException("No hay usuario autenticado");
    }
    return u;
}
```

Excepciones del dominio

Dos excepciones propias para separar los errores de negocio de los errores de sistema:

- `NotFoundException` → HTTP 404.
- `ForbiddenException` → HTTP 403.

Ampliación de `GlobalExceptionHandler`

Cambio a un archivo previo

El `GlobalExceptionHandler` creado en la Sesión 4 se amplió con dos nuevos métodos:

```
@ExceptionHandler(NotFoundException.class)
public ResponseEntity<Map<String, Object>> handleNotFound(
    NotFoundException e) { ... }

@ExceptionHandler(ForbiddenException.class)
public ResponseEntity<Map<String, Object>> handleForbidden(
    ForbiddenException e) { ... }
```

El formato de respuesta es el mismo que ya se usaba para `IllegalArgumentException`: `timestamp`, `status`, `error`, `message`.

`ChannelService`

Contiene la lógica de canales:

- `listPublic(category, q)`: lista canales públicos con filtros opcionales.
- `get(id)`: devuelve un canal con el campo `subscribedByMe` calculado para el usuario actual.
- `create(req)`: crea un canal con el usuario autenticado como dueño.
- `update(id, req)`: solo permitido al dueño o a un admin.
- `delete(id)`: solo permitido al dueño o a un admin.
- `listMine()` y `listSubscribed()`: filtros por usuario.

`SubscriptionService`

La suscripción se hace con una clave compuesta (`userId`, `channelId`). El servicio verifica primero si ya existe; si existe, retorna sin error.

`PostService`

Publicación y listado. En el listado, si el canal es privado y el usuario no es dueño ni suscriptor, lanza `ForbiddenException`. El borrado solo lo puede hacer el autor o un admin.

`ChannelController`

Concentra todos los endpoints anidados bajo `/api/channels`:

```
GET    /api/channels
GET    /api/channels/mine
GET    /api/channels/subscribed
GET    /api/channels/{id}
POST   /api/channels
PATCH  /api/channels/{id}
DELETE /api/channels/{id}

```

```
POST   /api/channels/{id}/subscribe
DELETE /api/channels/{id}/subscribe

GET    /api/channels/{id}/posts?page=0&size=20
POST   /api/channels/{id}/posts
```

5.3.4. Cambios a archivos previos

- `GlobalExceptionHandler.java`: se añadieron dos handlers.
- No se modificó `SecurityConfig.java`: la regla `anyRequest().authenticated()` ya cubría `/api/channels/**`.
- No se modificó `application.yml`.
- No se modificó ninguna entidad ni migración Flyway.

5.4. Verificación

5.4.1. Preparar variables

```
TOKEN=$(curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aaron@uam.mx","password":"password123"}' | jq -r .
     accessToken)

echo "Token: ${TOKEN:0:40}..."
```

Nota

El error más común es ejecutar el comando de login sin `TOKEN=$(...)` y perder el token. Sin la asignación, el token se imprime en pantalla pero no se guarda.

5.4.2. Crear, suscribirse y publicar

```
# Crear canal
CHANNEL_ID=$(curl -s -X POST http://localhost:8081/api/channels \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"Taller de Algoritmos","category":"ACADEMIC","
     publicChannel":true}' \
  | jq -r .id)

echo "Channel ID: $CHANNEL_ID"

# Suscribirse
curl -s -X POST "http://localhost:8081/api/channels/$CHANNEL_ID/
    subscribe" \
  -H "Authorization: Bearer $TOKEN" | jq

# Publicar
curl -s -X POST "http://localhost:8081/api/channels/$CHANNEL_ID/posts" \
```

```
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"content":"Primera publicacion"}' | jq
```

Resultado obtenido

```
{
  "subscribed": true,
  "channelId": "d5281a4b-8aa1-4dd4-a9e5-a86fb2425306",
  "subscriberCount": 1
}

{
  "id": "caba6493-5a2d-439c-b0ff-f589c7ff106c",
  "channelId": "d5281a4b-...",
  "channelName": "Taller de Algoritmos",
  "authorDisplayName": "Aaron Ramos",
  "content": "Primera publicacion",
  "createdAt": "2026-10-08T04:43:10.626Z"
}
```

5.4.3. Listar

```
curl -s "http://localhost:8081/api/channels/$CHANNEL_ID/posts?page=0&
    size=10" \
  -H "Authorization: Bearer $TOKEN" | jq

curl -s "http://localhost:8081/api/channels" \
  -H "Authorization: Bearer $TOKEN" | jq

curl -s "http://localhost:8081/api/channels/mine" \
  -H "Authorization: Bearer $TOKEN" | jq

curl -s "http://localhost:8081/api/channels/subscribed" \
  -H "Authorization: Bearer $TOKEN" | jq
```

5.4.4. Verificar seguridad

```
curl -i -X POST http://localhost:8081/api/channels \
  -H "Content-Type: application/json" \
  -d '{"name":"Sin auth","category":"OTHER"}'
```

Resultado esperado

`HTTP/1.1 403` sin cuerpo. La petición no trae token, por lo que Spring Security la rechaza antes de llegar al controller.

5.5. Resultado

Estado al cierre

- CRUD de canales funcional con control de acceso.
- Suscripciones idempotentes.
- Publicaciones paginadas.
- Filtros: por categoría, por búsqueda de texto, por canal, por usuario.
- Códigos HTTP correctos: 201 al crear, 204 al borrar, 403 sin token, 404 cuando no existe.

Pendiente para la siguiente sesión: conectar la publicación de posts con Kafka para emitir eventos y alimentar el sistema de notificaciones.

Capítulo 6

Sesión 5: Kafka productor, consumidor y notificaciones

6.1. Objetivo de la sesión

Convertir la publicación de avisos en un flujo de eventos distribuido:

- Publicar un evento `PostCreatedEvent` en Kafka cada vez que se crea un post.
- Consumir ese evento y notificar a los suscriptores del canal (excepto al autor).
- Mantener contadores de publicaciones no leídas por usuario en Redis.
- Exponer endpoints para consultar y limpiar esos contadores.

6.2. Contexto

La Sesión 5 dejó los endpoints de canales y publicaciones funcionando, pero **las notificaciones eran síncronas y locales**: el post se guardaba en PostgreSQL y la petición HTTP terminaba sin avisar a nadie. Este diseño contradice el modelo pub-sub descrito en el marco teórico (secciones 2.2 y 2.3.1).

Se necesitaba **desacoplar** la publicación de la notificación: el emisor del post no debe saber quién lo va a leer ni cuándo.

6.3. Desarrollo

6.3.1. Estructura creada

```
src/main/java/mx/uam/notiuam/
|- config/
|   ‘- KafkaConfig.java                  (definicion del topic)
|- event/
|   |- PostCreatedEvent.java             (payload)
|   |- PostEventPublisher.java           (productor)
|   ‘- PostNotificationConsumer.java     (consumidor)
|- service/
|   ‘- UnreadCounterService.java         (contadores en Redis)
```

```
‘- controller/
   ‘- MeController.java                    (endpoints de no leidos)
```

6.3.2. Decisiones técnicas

- **Topic con 3 particiones.** Permite paralelismo horizontal si más adelante se despliegan varias instancias del consumidor.
- **Clave de mensaje = `channelId`.** Todos los eventos de un mismo canal caen en la misma partición y se procesan en orden cronológico.
- **Consumer group dedicado** (`notiuam-notifications`). Permite escalar el consumidor sin perder eventos.
- **Contadores en Redis como hash.** Cada usuario tiene un hash `notiuam:unread:<userId>` donde cada campo es un `channelId` y el valor es el contador.
- **Auto-creación de topic.** El bean `NewTopic` hace que Spring Boot cree el topic al arrancar si no existe, sin depender de scripts externos.
- **No notificar al autor.** Aunque el autor sea suscriptor de su propio canal, no recibe notificación de sus propios posts.

6.3.3. Componentes clave

`KafkaConfig`

Define el topic como bean:

```java
@Configuration
public class KafkaConfig {
    public static final String POSTS_TOPIC = "notiuam.posts";

    @Bean
    public NewTopic postsTopic() {
        return TopicBuilder.name(POSTS_TOPIC)
                .partitions(3)
                .replicas(1)
                .build();
    }
}
```

`PostCreatedEvent`

Record inmutable con todo lo necesario para notificar: id del post, canal, autor y contenido.

```java
public record PostCreatedEvent(
        UUID postId,
        UUID channelId,
        String channelName,
        UUID authorId,
        String authorDisplayName,
        String content,
```

```
        Instant createdAt
) {}
```

PostEventPublisher

Envía el evento de forma asíncrona con `KafkaTemplate`. Usa `whenComplete` para loguear el resultado o el error sin bloquear la transacción.

```
public void publishPostCreated(PostCreatedEvent event) {
    kafkaTemplate.send(KafkaConfig.POSTS_TOPIC,
                       event.channelId().toString(),
                       event)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.error("Error publicando evento: {}", ex.
                        getMessage());
                } else {
                    log.info("Evento publicado: post={} partition={}
                        offset={}",
                             event.postId(),
                             result.getRecordMetadata().partition(),
                             result.getRecordMetadata().offset());
                }
            });
}
```

UnreadCounterService

Usa `StringRedisTemplate` para mantener contadores por usuario y canal.

```
public void increment(UUID userId, UUID channelId) {
    redis.opsForHash().increment(KEY_PREFIX + userId, channelId.toString
        (), 1);
}

public Map<String, Long> getAllCounts(UUID userId) {
    Map<Object, Object> raw = redis.opsForHash().entries(KEY_PREFIX +
        userId);
    return raw.entrySet().stream().collect(Collectors.toMap(
            e -> e.getKey().toString(),
            e -> Long.parseLong(e.getValue().toString())
    ));
}
```

PostNotificationConsumer

El listener de Kafka. Por cada evento:

1. Busca los suscriptores del canal.
2. Excluye al autor.
3. Incrementa el contador de no leídos de cada suscriptor en Redis.

4. Simula el envío de push (en la Sesión 6/7 se reemplazará con FCM real).

```
@KafkaListener(
        topics = KafkaConfig.POSTS_TOPIC,
        groupId = "notiuam-notifications"
)
public void onPostCreated(PostCreatedEvent event) {
    List<Subscription> subs = subscriptionRepository
            .findByChannelId(event.channelId());

    for (Subscription sub : subs) {
        if (sub.getUserId().equals(event.authorId())) continue;
        unreadCounters.increment(sub.getUserId(), event.channelId());
        log.info("[PUSH-SIMULADO] -> user={} | {}", sub.getUserId(),
           event.channelName());
    }
}
```

Modificación a `PostService`

Cambio a un archivo previo

El `PostService` de la Sesión 5 se modificó para publicar el evento después de guardar el post en PostgreSQL.
Se inyectó `PostEventPublisher` en el constructor y se añadió la llamada al publisher antes del `return`:

```
p = postRepository.save(p);

postEventPublisher.publishPostCreated(new PostCreatedEvent(
        p.getId(), c.getId(), c.getName(),
        author.getId(), author.getDisplayName(),
        p.getContent(), p.getCreatedAt()
));

return toResponse(p);
```

Con esto, el post se persiste primero y solo después se emite el evento. Si Kafka falla, el post sigue existiendo y el error solo se registra en los logs.

`MeController`

Expone los endpoints para consultar y limpiar los contadores:

```
@GetMapping("/unread")              // -> Map<channelId, count>
@DeleteMapping("/unread/{id}")      // -> 204 No Content
```

6.3.4. Cambios a archivos previos

- `PostService.java`: se añadió el campo `postEventPublisher` y una llamada a `publishPostCreated` dentro de `create()`.

- `application.yml`: ya tenía la configuración de Kafka desde la Sesión 3; no se tocó.
- `GlobalExceptionHandler`: sin cambios.

6.3.5. Problemas encontrados y solución

Método `create` duplicado en `PostService`

Al editar el archivo, el método nuevo se pegó debajo del viejo en lugar de reemplazarlo. La compilación falló con:

```
method create(UUID, PostCreateRequest) is already defined
```

Solución: eliminar el método viejo, dejar solo el nuevo que incluye la publicación a Kafka. Este error arrastró decenas de falsos positivos de Lombok (`cannot find symbol: method getId()`), que desaparecieron al corregir la duplicación.

`MeController` faltante

El endpoint `/api/me/unread` devolvía `HTTP 403` sin cuerpo. La causa: el archivo `MeController.java` no había sido creado, por lo que Spring Security interceptaba la ruta con `anyRequest().authenticated()` y la rechazaba.

Solución: crear el archivo, reiniciar el backend.

6.4. Verificación

6.4.1. Preparar dos usuarios

```
# Usuario A (autor)
TOKEN_A=$(curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"aaron@uam.mx","password":"password123"}' | jq -r .
     accessToken)

# Usuario B (suscriptor)
TOKEN_B=$(curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"betzabet@uam.mx","password":"password123"}' | jq -r .
     accessToken)
```

6.4.2. Publicar y observar los logs

Crear canal, suscribir a ambos usuarios, publicar con el usuario A:

```
CHANNEL_ID=$(curl -s -X POST http://localhost:8081/api/channels \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN_A" \
  -d '{"name":"Avisos de Ingenieria","category":"ADMINISTRATIVE","
     publicChannel":true}' \
  | jq -r .id)

```

```
curl -s -X POST "http://localhost:8081/api/channels/$CHANNEL_ID/
    subscribe" \
  -H "Authorization: Bearer $TOKEN_A" > /dev/null

curl -s -X POST "http://localhost:8081/api/channels/$CHANNEL_ID/
    subscribe" \
  -H "Authorization: Bearer $TOKEN_B" > /dev/null

curl -s -X POST "http://localhost:8081/api/channels/$CHANNEL_ID/posts" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN_A" \
  -d '{"content":"Prueba de Kafka"}' | jq
```

Logs obtenidos

```
Evento publicado: post=ed496ba0... canal=f4c3bbdd... partition=1
    offset=0
Kafka event recibido: post=ed496ba0... canal=Avisos de Ingenieria
[PUSH-SIMULADO] -> user=b8cc6002... | Avisos de Ingenieria | "Prueba
    de Kafka"
Notificaciones procesadas: 1 de 2 suscriptores
```

El `1 de 2` confirma que el autor (Aaron) no se notifica a sí mismo, y que el otro suscriptor (Betzabet) sí recibe el aviso.

6.4.3. Consultar y limpiar contadores

```
# Betzabet consulta sus no leidos
curl -s http://localhost:8081/api/me/unread \
  -H "Authorization: Bearer $TOKEN_B" | jq
# -> { "f4c3bbdd-...": 1 }

# Aaron no deberia tener nada
curl -s http://localhost:8081/api/me/unread \
  -H "Authorization: Bearer $TOKEN_A" | jq
# -> {}

# Betzabet marca como leido
curl -s -X DELETE "http://localhost:8081/api/me/unread/$CHANNEL_ID" \
  -H "Authorization: Bearer $TOKEN_B" | jq

# Ya no debe aparecer
curl -s http://localhost:8081/api/me/unread \
  -H "Authorization: Bearer $TOKEN_B" | jq
# -> {}
```

6.4.4. Verificación en Redis

```
docker exec -it notiuam-redis redis-cli

# Dentro del CLI
```

```
KEYS notiuam:unread:*
HGETALL notiuam:unread:<userId>
exit
```

6.4.5. Verificación en Kafka UI

En http://localhost:8080:

- **Topics** → `notiuam.posts` → 3 particiones con mensajes.
- **Consumer Groups** → `notiuam-notifications` → offsets por partición con `LAG = 0`.

6.5. Resultado

Estado al cierre

- Publicación de posts → evento en Kafka → consumer → contador en Redis, todo automático.
- Arquitectura pub-sub real (desacoplamiento entre productor y consumidor).
- Persistencia doble: PostgreSQL para los posts, Redis para los contadores.
- Endpoints `/api/me/unread` funcionales.
- Base lista para integrar notificaciones push reales con FCM en la Sesión 6.

Pendiente para la siguiente sesión: cliente móvil en Flutter (login, exploración de canales, suscripción, publicación, contadores, notificaciones push reales).

Capítulo 7

Sesión 6a: Setup del cliente Flutter y autenticación

7.1. Objetivo de la sesión

Crear el cliente móvil multiplataforma en Flutter con:

- Estructura de carpetas escalable (core, models, state, screens).
- Cliente HTTP con inyección automática del JWT.
- Almacenamiento seguro del token en el sistema operativo.
- Manejo de estado con `provider` (`ChangeNotifier`).
- Pantallas de splash, login y registro funcionales contra el backend real.

7.2. Contexto

Al cierre de la Sesión 6, el backend exponía todos los endpoints necesarios (autenticación, canales, suscripciones, publicaciones, contadores). El siguiente paso natural, según la Fase 6 del protocolo, era construir la aplicación móvil multiplataforma con Flutter que consumiera esa API.

Se optó por dividir la Sesión 6 en cuatro sub-sesiones para que cada una entregue algo verificable:

1. **6a (este capítulo):** setup + autenticación.
2. **6b:** listado, búsqueda, detalle y creación de canales.
3. **6c:** publicaciones dentro del canal y contadores de no leídos.
4. **6d:** notificaciones push reales con Firebase Cloud Messaging.

7.3. Desarrollo

7.3.1. Generación del proyecto

Se creó el proyecto Flutter con organización `mx.uam`:

```
cd ~/Escritorio/notiuam
flutter create mobile --org mx.uam --project-name notiuam_mobile
```

7.3.2. Dependencias

Se configuraron en `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  dio: ^5.7.0                          # Cliente HTTP
  flutter_secure_storage: ^9.2.2       # Token seguro en Keychain /
      EncryptedSharedPreferences
  provider: ^6.1.2                     # Manejo de estado
  intl: ^0.19.0                        # Formato de fechas

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
```

7.3.3. Estructura de carpetas

```
mobile/lib/
|- main.dart                           (punto de entrada, MultiProvider)
|- core/
|  |- constants.dart                   (ApiConfig: baseUrl segun
   plataforma)
|  |- token_storage.dart               (guardado seguro del JWT)
|  `- api_client.dart                  (Dio + interceptor de
   autenticacion)
|- models/
|  |- user.dart
|  |- auth_response.dart
|  |- channel.dart
|  `- post.dart
|- state/
|  `- auth_provider.dart               (ChangeNotifier de autenticacion)
`- screens/
   |- splash_screen.dart
   |- login_screen.dart
   |- register_screen.dart
   `- home_shell.dart                  (barra de navegacion inferior)
```

7.3.4. Decisiones técnicas

- **`dio` en lugar de `http`.** Dio ofrece interceptores, manejo estructurado de errores (`DioException`) y mejor configuración de timeouts.

- **Interceptor de autenticación.** El `ApiClient` lee el token desde `TokenStorage` en cada petición y lo añade al header `Authorization: Bearer <token>` automáticamente. Las pantallas no se preocupan por el token.
- **`flutter_secure_storage` en lugar de `shared_preferences`.** Guarda el JWT en el Keychain de iOS o en `EncryptedSharedPreferences` de Android. Nunca en texto plano.
- **`provider` con `ChangeNotifier`.** Patrón estándar de la comunidad Flutter. Alternativa a `bloc` que se descartó por ser más verboso para el tamaño del proyecto.
- **`ProxyProvider` para dependencias encadenadas.** El `ApiClient` depende del `TokenStorage`, y el `AuthProvider` del `ApiClient`. Se usan `ProxyProvider` y `ChangeNotifierProxyProvider` para inyectarlos sin duplicar instancias.

7.3.5. Detección de plataforma para la URL del backend

El emulador de Android usa `10.0.2.2` para referirse al host, mientras que Linux/iOS usan `localhost`. El archivo `constants.dart` encapsula la lógica:

```
class ApiConfig {
  static String get baseUrl {
    if (Platform.isAndroid) {
      return 'http://10.0.2.2:8081';
    }
    return 'http://localhost:8081';
  }
  static const Duration timeout = Duration(seconds: 15);
}
```

7.3.6. Configuración Android

Se editó `android/app/src/main/AndroidManifest.xml` para:

- Añadir el permiso `android.permission.INTERNET`.
- Habilitar `android:usesCleartextTraffic="true"` (necesario porque el backend sirve HTTP en desarrollo, no HTTPS).

Advertencia

`usesCleartextTraffic="true"` solo es aceptable en `debug`. En la versión de producción habrá que exponer el backend por HTTPS con un certificado válido.

7.3.7. Cambios a archivos previos

Ninguno. Esta es una sub-sesión completamente nueva.

7.3.8. Problemas encontrados y solución

Falta el SDK de Android

Al primer `flutter run` apareció:

```
SDK location not found. Define a valid SDK location with an
ANDROID_HOME environment variable or by setting the sdk.dir path...
```

Solución: instalar Android Studio por Snap (`sudo snap install android-studio -classic`), configurar el SDK siguiendo el asistente, y añadir las variables al shell:

```
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin
```

Clang++ no encontraba `libstdc++`

Al intentar compilar para Linux desktop:

```
/usr/bin/ld: no se puede encontrar -lstdc++: No existe el archivo o el
   directorio
```

El diagnóstico con `clang++ -v` reveló que clang usa por defecto GCC 14 (el más reciente) mientras que `g++` apunta a GCC 13. Faltaba el paquete `libstdc++-14-dev`.

Solución:

```
sudo apt install -y libstdc++-14-dev
```

Falta `libsecret-1-dev`

El plugin `flutter_secure_storage_linux` requiere `libsecret` para acceder al llavero del sistema:

```
The following required packages were not found:
  - libsecret-1>=0.18.4
```

Solución:

```
sudo apt install -y libsecret-1-dev
```

Archivos `api_client.dart` y `token_storage.dart` con contenido duplicado

Ambos archivos contenían el mismo bloque de `constants.dart` por un error de copiado. La compilación falló con errores de tipo `'ApiClient' isn't a type` y `'TokenStorage' isn't a type`.

Solución: sobrescribir ambos archivos con el contenido correcto mediante una escritura completa del archivo para garantizar que no quedaran partes duplicadas.

7.4. Verificación

7.4.1. Arranque

```
cd ~/Escritorio/notiuam/mobile
flutter clean
flutter run -d linux
```

La primera compilación tarda 1–2 minutos. Al terminar se abre una ventana nativa con la app NotiUAM.

7.4.2. Flujo verificado

1. **Splash** visible 1 segundo mientras `AuthProvider.bootstrap()` verifica si hay token guardado.
2. **Login** con `aaron@uam.mx` / `password123` → entra al `HomeShell`.
3. **HomeShell** con 4 pestañas: Explorar, Mis canales, Suscritos, Perfil.
4. **Logout** desde el AppBar → vuelve al login y limpia el token del storage.
5. **Persistencia:** cerrar la app y reabrirla mantiene la sesión activa (token en Keychain / EncryptedSharedPreferences).

7.5. Resultado

Estado al cierre

- Proyecto Flutter funcional en Linux desktop (y listo para Android).
- Cliente HTTP con JWT automático.
- Almacenamiento seguro del token.
- Login y registro funcionales contra el backend real.
- Arquitectura de estado escalable para las próximas sub-sesiones.

Pendiente para la siguiente sub-sesión: reemplazar los placeholders del `HomeShell` por pantallas reales de canales.

Capítulo 8

Sesión 6b: Listado, búsqueda y creación de canales

8.1. Objetivo de la sesión

Sustituir las cuatro pestañas placeholder del `HomeShell` por pantallas funcionales:

- **Explorar:** lista de canales públicos con búsqueda y pull-to-refresh.
- **Detalle del canal:** información completa + botón de suscripción.
- **Crear canal:** formulario con validación, selector de categoría y feedback de errores.
- **Mis canales** y **Suscritos:** listados filtrados por el usuario autenticado.
- **Perfil:** nombre del usuario autenticado.

8.2. Contexto

La Sesión 7 dejó el login funcional, pero al entrar al `HomeShell` solo se veían cuatro textos de "en construcción". El objetivo ahora es conectar la interfaz con los endpoints `/api/channels/**` que ya funcionaban desde la Sesión 5.

8.3. Desarrollo

8.3.1. Archivos nuevos

```
mobile/lib/
|- state/
|  ‘- channel_provider.dart          (NUEVO)
‘- screens/
   |- explore_screen.dart            (NUEVO)
   |- channel_detail_screen.dart     (NUEVO)
   ‘- create_channel_screen.dart     (NUEVO)
```

8.3.2. Archivos modificados

- `lib/screens/home_shell.dart`: se reemplazaron los `_Placeholder` por pantallas reales.
- `lib/main.dart`: se añadió el `ChannelProvider` al árbol de `MultiProvider`.

8.3.3. Decisiones técnicas

- **Un solo `ChannelProvider` para todo el dominio de canales.** Mantiene tres listas (`publicChannels`, `mine`, `subscribed`) y expone métodos de carga por separado. Evita crear un provider por pestaña.
- **Recarga tras cada mutación.** Al crear un canal o cambiar una suscripción, el provider recarga automáticamente las listas afectadas. Esto mantiene la UI sincronizada sin estado duplicado.
- **Iconos por categoría.** La pestaña Explorar muestra un icono y color distinto según la categoría del canal, lo que mejora el reconocimiento visual.
- **Validación en formularios con `GlobalKey<FormState>`.** Los campos del formulario de creación se validan antes de enviar la petición.
- **Feedback de error con `SnackBar`.** Los errores de suscripción se muestran como snack bar en lugar de bloquear la pantalla; los errores del formulario se muestran inline.

8.3.4. Componentes clave

`ChannelProvider`

Métodos públicos:

```
Future<void> loadPublic({String? query});
Future<void> loadMine();
Future<void> loadSubscribed();
Future<Channel> getById(String id);
Future<Channel> create({required String name, String? description,
   required String category});
Future<void> toggleSubscription(Channel channel);
```

Estado expuesto:

```
List<Channel> get publicChannels;
List<Channel> get mine;
List<Channel> get subscribed;
bool get loading;
String? get error;
```

`ExploreScreen`

Estructura visual:

- Barra de búsqueda fija en la parte superior con botón de limpiar.
- `LinearProgressIndicator` visible solo mientras `loading`.
- `RefreshIndicator` para pull-to-refresh.

- Lista de `Card` con `ListTile` por cada canal (icono, nombre, descripción, contador de suscriptores, indicador de suscripción).
- `FloatingActionButton.extended` con acción "Crear canal".

La búsqueda dispara `loadPublic(query: <texto>)` que el backend traduce a un `LIKE %texto%`.

`ChannelDetailScreen`

Recibe un `channelId` por parámetro. En `initState` carga el canal con `getById`. La pantalla muestra:

- Nombre y `Chip` con la categoría.
- Descripción.
- Autor y número de suscriptores.
- Botón principal cuyo texto y acción cambian según `subscribedByMe`:
  - `Suscribirse` → `POST /api/channels/{id}/subscribe`.
  - `Desuscribirse` → `DELETE /api/channels/{id}/subscribe`.
- Marcador para la sección de publicaciones (se completa en la Sesión 9).

`CreateChannelScreen`

Formulario con:

- Campo de nombre (mínimo 3 caracteres).
- Campo de descripción (opcional, multilínea).
- `DropdownButtonFormField<String>` con las seis categorías del dominio.
- Botón de envío con estado de carga.

Al enviar, llama a `ChannelProvider.create(...)` y hace `pop()` para volver a la pantalla anterior. El nuevo canal aparece automáticamente porque `create()` llama internamente a `loadMine()`.

`HomeShell` actualizado

Se reemplazaron los placeholders. La estructura de pestañas ahora es:

```
final pages = [
  const ExploreScreen(),
  const _MineScreen(),
  const _SubscribedScreen(),
  _ProfileScreen(displayName: auth.user?.displayName ?? 'Usuario'),
];
```

`_MineScreen` y `_SubscribedScreen` son `StatefulWidget` que cargan sus listas en `initState` mediante `WidgetsBinding.instance.addPostFrameCallback`.

8.3.5. Cambios a archivos previos

- `home_shell.dart`: se eliminó la clase `_Placeholder` y se añadieron `_MineScreen`, `_SubscribedScreen` y `_ProfileScreen`.
- `main.dart`: se añadió el import `state/channel_provider.dart` y un `ChangeNotifierProxyProvider<ApiClient, ChannelProvider>` al `MultiProvider`.

8.4. Verificación

8.4.1. Compilación

```
cd ~/Escritorio/notiuam/mobile
flutter run -d linux
```

Debe compilar sin errores y abrir la ventana de la app.

8.4.2. Flujo verificado

Con el backend corriendo en `localhost:8081`:

1. Login con `aaron@uam.mx` / `password123`.
2. **Pestaña Explorar** carga la lista de canales públicos (incluyendo los creados por curl en sesiones anteriores).
3. **Búsqueda:** escribir un texto y pulsar Enter filtra los canales por nombre.
4. **Tocar un canal** abre el detalle con descripción, autor y contador de suscriptores.
5. **Pulsar Suscribirse** cambia el botón a "Desuscribirse" y suma 1 al contador.
6. **FAB "Crear canal"** abre el formulario. Al enviarlo:
   - Vuelve a la pantalla anterior.
   - El nuevo canal aparece en la lista de Explorar.
   - El nuevo canal aparece en "Mis canales".
7. **Pestaña Suscritos** muestra los canales a los que el usuario se ha suscrito.
8. **Pestaña Perfil** muestra el nombre del usuario autenticado.

8.5. Resultado

Estado al cierre

- Navegación entre pantallas funcional.
- Listado de canales públicos, filtrado por búsqueda.
- Creación de canales desde la app.
- Suscripción y desuscripción desde el detalle.

- Sincronización automática de listas tras cada mutación.

Pendiente para la siguiente sub-sesión: añadir la sección de publicaciones dentro del detalle del canal, el botón para publicar desde la app, y mostrar los contadores de no leídos por canal.

Capítulo 9

Sesión 6c: Publicaciones y contadores de no leídos en Flutter

9.1. Objetivo de la sesión

Completar el cliente móvil con:

- Visualización de las publicaciones dentro del canal.
- Publicación de avisos desde la app.
- Consulta y despliegue de contadores de no leídos por canal.
- Marcado automático como leído al abrir un canal.
- Badge numérico en la pestaña "Suscritos" con el total de no leídos.

9.2. Contexto

La Sesión 8 dejó el listado, búsqueda, detalle y creación de canales funcional. Sin embargo, el detalle del canal solo mostraba un placeholder en lugar de las publicaciones. El backend (Sesiones 5 y 6) ya exponía endpoints de posts y contadores, así que el trabajo era exclusivamente del cliente.

9.3. Desarrollo

9.3.1. Archivos nuevos

```
mobile/lib/state/
|- post_provider.dart           (NUEVO)
‘- unread_provider.dart         (NUEVO)
```

9.3.2. Archivos modificados

- `lib/main.dart`: se registraron `PostProvider` y `UnreadProvider` en el `MultiProvider`.
- `lib/screens/channel_detail_screen.dart`: se reemplazo el placeholder por la lista de publicaciones, la caja para publicar y la logica de marcado como leido.

- `lib/screens/home_shell.dart`: se agrego el badge numerico en la pestana "Suscritos".

9.3.3. Decisiones tecnicas

- **Un `PostProvider` que cachea por canal.** Mantiene un `Map<String, List<Post>>` indexado por `channelId`. Esto permite cambiar entre canales sin rehacer peticiones.
- **Carga optimista al publicar.** Cuando el usuario publica, el post se inserta inmediatamente al inicio de la lista local sin esperar a recargar todo el canal.
- **Borrado local tras exito del backend.** El `delete` solo modifica el estado local despues de que el servidor responde 204.
- **Contadores en Redis como fuente de verdad.** El `UnreadProvider` solo consulta `/api/me/unread` y expone el `Map<String, int>` resultante.
- **Marcado como leido al entrar.** Al cargar el detalle del canal, se llama automaticamente a `markRead(channelId)`.

9.3.4. Componentes clave

`PostProvider`

```
class PostProvider extends ChangeNotifier {
  final Map<String, List<Post>> _byChannel = {};
  final Map<String, bool> _loading = {};

  List<Post> postsOf(String channelId);
  bool isLoading(String channelId);

  Future<void> load(String channelId, {bool refresh = false});
  Future<Post> create(String channelId, String content);
  Future<void> delete(String channelId, String postId);
}
```

`UnreadProvider`

```
class UnreadProvider extends ChangeNotifier {
  Map<String, int> _counts = {};

  int get total => _counts.values.fold(0, (a, b) => a + b);
  int countFor(String channelId) => _counts[channelId] ?? 0;

  Future<void> refresh();
  Future<void> markRead(String channelId);
  void clear();
}
```

Pantalla de detalle del canal

Estructura visual:

- Encabezado: nombre, chip de categoria, badge de rol del owner (se anade en la Fase E), descripcion, autor, contador de suscriptores y boton Suscribirse/Desuscribirse.
- Caja de publicacion: `TextField` multilinea + boton de envio.
- Lista de publicaciones con `RefreshIndicator` y paginacion inicial de 50 elementos.

9.3.5. Cambios a archivos previos

- `main.dart`: `MultiProvider` ampliado con dos providers.
- `channel_detail_screen.dart`: reemplazo completo.
- `home_shell.dart`: badge en la pestana.

9.3.6. Problemas encontrados y solucion

Locale de fechas no inicializado

Al abrir el detalle del canal, la app mostraba:

```
LocaleDataException: Locale data has not been initialized,
call initializeDateFormatting(<locale>).
```

Solucion: inicializar el locale en `main()`:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDateFormatting('es', null);
  runApp(const NotiuamApp());
}
```

9.4. Verificacion

Con el backend corriendo en `localhost:8081` y una sesion iniciada:

1. Entrar a cualquier canal.
2. Ver el listado de publicaciones anteriores.
3. Publicar un aviso nuevo desde la caja de texto.
4. Confirmar que aparece al instante en la lista.
5. Abrir el canal desde el usuario destinatario y verificar que el badge de no leidos se limpia.

9.5. Resultado

Estado al cierre

- Publicaciones visibles y creables desde la app.
- Contadores de no leidos consumidos desde Redis.
- Badge numerico en la barra inferior.
- Marcado automatico como leido al abrir el canal.
- Base para la moderacion directa (Fase D5) y para push real (Fase F).

Capítulo 10

Rediseno UX/UI con paleta UAM-C

10.1. Objetivo

Mejorar el acabado visual de la primera version del cliente:

- Definir una paleta inspirada en el naranja institucional de la UAM Cuajimalpa.
- Centralizar los estilos en un unico `ThemeData`.
- Eliminar el problema de iconos de Material que se renderizaban como glifos de fuentes CJK.
- Redisenar las pantallas de login y registro con el nuevo estilo.

10.2. Contexto

Al llegar a la Sesion 8, la app funcionaba pero se veia generica. Ademas, los iconos de Material Design aparecian como caracteres raros en la barra de navegacion inferior, en el campo de busqueda y en el boton de logout.

10.3. Desarrollo

10.3.1. Diagnostico del problema de iconos

Los glifos "raros" no eran caracteres chinos insertados en el codigo. Eran iconos de Material Design que no podian cargarse porque faltaba la directiva `uses-material-design: true` en el `pubspec.yaml`. Al no encontrarlos, Flutter caia en fuentes de respaldo del sistema (CJK), lo que producia los glifos incorrectos.

10.3.2. Correccion del `pubspec.yaml`

```
1 name: notiuam_mobile
2 description: "Plataforma distribuida de comunicacion y alertas
     comunitarias."
3 publish_to: 'none'
4 version: 1.0.0+1
5
6 environment:
```

```
  sdk: ^3.13.4

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  dio: ^5.7.0
  flutter_secure_storage: ^9.2.2
  provider: ^6.1.2
  intl: ^0.19.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

10.3.3. Paleta institucional

Se definio una paleta que parte del naranja institucional:

Rol	Hex	Uso
`orange`	#E85D04	Botones, AppBar
`orangeDark`	#B84700	Estado pressed
`orangeLight`	#F48C06	Chips, badges
`charcoal`	#2B2B2B	Texto principal
`cream`	#FAFAF7	Fondo
`success`	#2A9D8F	Acciones positivas
`danger`	#D62828	Reportes, eliminar
`info`	#457B9D	Informacion

Cuadro 10.1: Paleta UAM-C utilizada en el cliente Flutter

10.3.4. Centralizacion del tema

Se creo `lib/core/theme.dart` con dos clases: `NotiuamColors` (constantes) y `NotiuamTheme` (constructor del `ThemeData`).

```
class NotiuamTheme {
  static ThemeData light() {
    final base = ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(
        seedColor: NotiuamColors.orange,
        primary: NotiuamColors.orange,
        brightness: Brightness.light,
      ),
    );

```

```
    return base.copyWith(
      scaffoldBackgroundColor: NotiuamColors.cream,
      appBarTheme: const AppBarTheme(
        backgroundColor: NotiuamColors.orange,
        foregroundColor: Colors.white,
      ),
      filledButtonTheme: ...,
      outlinedButtonTheme: ...,
      inputDecorationTheme: ...,
      chipTheme: ...,
      navigationBarTheme: ...,
    );
  }
}
```

10.3.5. Rediseno de pantallas

Login

Se agrego un bloque de logo cuadrado con la letra N, subtitulo descriptivo, inputs con iconos, caja de error con fondo rosado y boton de altura fija con estado de carga.

Registro

Se anadio un encabezado con titulo y subtitulo, campos con iconos y limitacion de ancho a 420 px con `ConstrainedBox`.

10.3.6. Cambios a archivos previos

- `pubspec.yaml`: se reescribio agregando `uses-material-design: true`.
- `main.dart`: se cambio `theme: ThemeData(...)` por `theme: NotiuamTheme.light()`.
- `login_screen.dart` y `register_screen.dart`: rediseno completo.
- `explore_screen.dart`: se ajustaron los colores del metodo `_categoryIcon`.

10.4. Verificacion

- Los iconos de la barra inferior se ven como brujula, carpeta, marcador y persona.
- El AppBar y los botones principales usan el naranja institucional.
- Las tarjetas usan fondo blanco sobre crema.
- Los campos de texto tienen borde de foco naranja.

10.5. Resultado

Estado al cierre

- Tema visual coherente en toda la app.
- Iconos de Material correctos.
- Base para las pantallas de administracion y moderacion.

Capítulo 11

Backend: reportes, votacion y roles

11.1. Objetivo

Extender el backend con:

- Sistema de reportes para publicaciones y canales.
- Votacion colegiada de administradores (3 votos para resolver).
- Gestion de roles con jerarquia (STUDENT, PROFESSOR, ADMIN, SUPERADMIN).
- Baneo y desbaneo de usuarios.
- Log de auditoria de acciones administrativas.
- Moderacion directa por dueno del canal.

11.2. Contexto

Hasta la Sesion 6, el backend tenia el dominio basico: usuarios, canales, suscripciones, publicaciones y notificaciones. No existia ningun mecanismo de moderacion ni distincion real entre roles mas alla del valor del enum.

Se decidio introducir tres reglas clave:

1. Solo el SUPERADMIN puede promover/degradar libremente (excepto quitar SUPERADMIN).
2. Los ADMIN solo pueden promover estudiantes a PROFESSOR o ADMIN.
3. Los duenos de un canal pueden moderar sus publicaciones sin votacion.

11.3. Desarrollo

11.3.1. Migracion V2

Se creo `V2__reports_and_roles.sql` con tres tablas:

- `reports`: un reporte por (target, reporter).
- `report_votes`: un voto por (report, admin).

- `admin_action_log`: bitacora de acciones administrativas.

```
CREATE TABLE reports (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    target_type  VARCHAR(20)  NOT NULL CHECK (target_type IN ('POST', '
       CHANNEL')),
    target_id    UUID         NOT NULL,
    reporter_id  UUID         NOT NULL REFERENCES users(id) ON DELETE
       CASCADE,
    reason       VARCHAR(50)  NOT NULL,
    details      VARCHAR(500),
    status       VARCHAR(30)  NOT NULL DEFAULT 'PENDING',
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    resolved_at  TIMESTAMPTZ,
    resolved_by  UUID REFERENCES users(id) ON DELETE SET NULL,
    UNIQUE (target_type, target_id, reporter_id)
);

CREATE TABLE report_votes (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id   UUID NOT NULL REFERENCES reports(id) ON DELETE CASCADE,
    admin_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    decision    VARCHAR(20) NOT NULL CHECK (decision IN ('KEEP', 'DELETE
       ')),
    comment     VARCHAR(500),
    voted_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (report_id, admin_id)
);

CREATE TABLE admin_action_log (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    action       VARCHAR(50) NOT NULL,
    target_type  VARCHAR(20),
    target_id    UUID,
    details      VARCHAR(500),
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.3.2. Enumeraciones

```
public enum Role { STUDENT, PROFESSOR, ADMIN, SUPERADMIN }
public enum ReportTargetType { POST, CHANNEL }
public enum ReportReason { SPAM, INAPPROPRIATE, MISINFORMATION,
   HARASSMENT, OTHER }
public enum ReportStatus { PENDING, RESOLVED_KEEP, RESOLVED_DELETE }
public enum VoteDecision { KEEP, DELETE }
```

11.3.3. Entidades JPA

Se crearon `Report`, `ReportVote` y `AdminActionLog` siguiendo el mismo patron que las entidades previas: UUID como clave primaria, `@CreatedDate` para auditoria.

11.3.4. Repositorios

```
public interface ReportRepository extends JpaRepository<Report, UUID> {
    Page<Report> findByStatusOrderByCreatedAtDesc(ReportStatus status,
        Pageable p);
    Page<Report> findAllByOrderByCreatedAtDesc(Pageable p);
    List<Report> findByTargetTypeAndTargetIdAndStatus(
        ReportTargetType type, UUID targetId, ReportStatus status);
    long countByStatus(ReportStatus status);
    boolean existsByTargetTypeAndTargetIdAndReporterId(
        ReportTargetType type, UUID targetId, UUID reporterId);
}

public interface ReportVoteRepository extends JpaRepository<ReportVote,
    UUID> {
    List<ReportVote> findByReportId(UUID reportId);
    Optional<ReportVote> findByReportIdAndAdminId(UUID reportId, UUID
        adminId);
    long countByReportIdAndDecision(UUID reportId, VoteDecision decision
        );
    long countByReportId(UUID reportId);
}

public interface AdminActionLogRepository extends JpaRepository<
    AdminActionLog, UUID> {
    Page<AdminActionLog> findAllByOrderByCreatedAtDesc(Pageable p);
    Page<AdminActionLog> findByAdminIdOrderByCreatedAtDesc(UUID adminId,
        Pageable p);
}
```

11.3.5. ReportService

Es el nucleo del sistema de moderacion colegiada. Reglas:

1. Cualquier usuario autenticado puede reportar.
2. Un mismo usuario no puede reportar dos veces el mismo contenido.
3. Un admin no puede votar en un reporte que el mismo creo.
4. Cada admin vota una sola vez por reporte.
5. Al alcanzar 3 votos, se resuelve por mayoria simple.
6. Si gana DELETE, el contenido se elimina automaticamente.

```
public static final int VOTES_REQUIRED = 3;

private void resolveIfComplete(Report report) {
    long totalVotes = voteRepository.countByReportId(report.getId());
    if (totalVotes < VOTES_REQUIRED) return;

    long deleteVotes = voteRepository.countByReportIdAndDecision(
```

```
            report.getId(), VoteDecision.DELETE);
    long keepVotes = voteRepository.countByReportIdAndDecision(
            report.getId(), VoteDecision.KEEP);

    ReportStatus newStatus;
    if (deleteVotes > keepVotes) {
        newStatus = ReportStatus.RESOLVED_DELETE;
        applyDeletion(report);
    } else {
        newStatus = ReportStatus.RESOLVED_KEEP;
    }

    report.setStatus(newStatus);
    report.setResolvedAt(Instant.now());
    reportRepository.save(report);
}
```

11.3.6. `AdminUserService`

Encapsula la logica de cambio de rol y ban. Reglas aplicadas en orden:

1. Nadie puede cambiarse su propio rol.
2. Nunca se puede quitar el rol SUPERADMIN.
3. Solo SUPERADMIN puede nombrar a otro SUPERADMIN.
4. ADMIN solo puede promover STUDENT a PROFESSOR o ADMIN.
5. Solo SUPERADMIN puede degradar.

```
public UserSummaryResponse changeRole(UUID targetUserId, Role newRole) {
    var me = currentUser.require();
    UUID myId = me.id();
    Role myRole = Role.valueOf(me.role());

    if (myId.equals(targetUserId))
        throw new ForbiddenException("No puedes cambiar tu propio rol");

    User target = userRepository.findById(targetUserId)
        .orElseThrow(() -> new NotFoundException("Usuario no encontrado"
           ));

    Role currentRole = target.getRole();

    if (currentRole == Role.SUPERADMIN && newRole != Role.SUPERADMIN)
        throw new ForbiddenException("No se puede degradar a un
           SUPERADMIN");

    if (newRole == Role.SUPERADMIN && myRole != Role.SUPERADMIN)
        throw new ForbiddenException("Solo un SUPERADMIN puede nombrar a
            otro SUPERADMIN");

```

```
20      if (myRole == Role.ADMIN) {
21          if (currentRole != Role.STUDENT)
22              throw new ForbiddenException("Como ADMIN solo puedes
                    promover estudiantes");
23          if (newRole != Role.PROFESSOR && newRole != Role.ADMIN)
24              throw new ForbiddenException("Como ADMIN solo puedes
                    promover a PROFESSOR o ADMIN");
25      }
26
27      if (myRole != Role.SUPERADMIN && level(newRole) < level(currentRole)
            )
28          throw new ForbiddenException("Solo un SUPERADMIN puede degradar
                usuarios");
29
30      target.setRole(newRole);
31      userRepository.save(target);
32      // ...
33      return toResponse(target);
34  }
```

11.3.7. Moderacion directa por dueno del canal

En `PostService.delete()` se anadio la regla que permite al dueno del canal (incluyendo profesores) borrar publicaciones ajenas sin necesidad de votacion.

```
1   @Transactional
2   public void delete(UUID postId) {
3       Post p = postRepository.findById(postId)
4           .orElseThrow(() -> new NotFoundException("Publicacion no
                encontrada"));
5
6       var me = currentUser.require();
7       UUID myId = me.id();
8       Role myRole = Role.valueOf(me.role());
9
10      UUID authorId = p.getAuthor().getId();
11      UUID channelOwnerId = p.getChannel().getOwner().getId();
12
13      boolean isAuthor = authorId.equals(myId);
14      boolean isChannelOwner = channelOwnerId.equals(myId);
15      boolean isAdmin = myRole == Role.ADMIN || myRole == Role.SUPERADMIN;
16
17      if (!isAuthor && !isChannelOwner && !isAdmin) {
18          throw new ForbiddenException(
19              "Solo el autor, el dueno del canal o un moderador pueden
                    eliminar");
20      }
21
22      if (!isAuthor) {
23          User admin = userRepository.findById(myId).orElseThrow();
24          String reason = isChannelOwner
25              ? "Moderacion del dueno del canal"
26              : "Moderacion administrativa";
27          adminLog.log(admin, "DELETE_POST", ReportTargetType.POST,
```

```
            postId, reason + ": " + p.getChannel().getName());
    }

    postRepository.delete(p);
}
```

11.3.8. Endpoints

```
POST   /api/reports                          (cualquier usuario)
GET    /api/admin/reports?status=...         (ADMIN, SUPERADMIN)
GET    /api/admin/reports/{id}               (ADMIN, SUPERADMIN)
POST   /api/admin/reports/{id}/vote          (ADMIN, SUPERADMIN)
GET    /api/admin/users?q=...                (ADMIN, SUPERADMIN)
GET    /api/admin/users/{id}                 (ADMIN, SUPERADMIN)
PATCH  /api/admin/users/{id}/role            (ADMIN, SUPERADMIN con
   reglas)
PATCH  /api/admin/users/{id}/status          (solo SUPERADMIN)
GET    /api/admin/audit                      (ADMIN propio / SUPERADMIN
   todo)
DELETE /api/posts/{id}                       (autor, dueno, moderador)
```

11.3.9. Habilitacion de `@PreAuthorize`

Se agrego `@EnableMethodSecurity` a `SecurityConfig` para que las anotaciones de metodo sean respetadas.

11.3.10. Problemas encontrados y solucion

`LazyInitializationException` en el endpoint de auditoria

El endpoint `/api/admin/audit` devolvia 403 en lugar de 200. El log revelo una excepcion oculta:

```
org.hibernate.LazyInitializationException: Could not initialize proxy
[mx.uam.notiuam.domain.User#...] - no session
    at ...AdminAuditController.lambda$list$0(AdminAuditController.java
       :41)
```

El mapeo de entidades a DTO se hacia en el controller, fuera de la transaccion, y al acceder a `log.getAdmin().getDisplayName()` la sesion JPA ya estaba cerrada.

Solucion: mover el mapeo al servicio (`AdminLogService`), dentro del `@Transactional(readOnly = true)`.

11.4. Verificacion

11.4.1. Reglas de rol

- SUPERADMIN promueve y degrada sin restricciones (salvo quitar SUPERADMIN).
- ADMIN solo promueve estudiantes.
- ADMIN no puede banear.
- Un usuario baneado no puede iniciar sesion.

11.4.2. Flujo de reporte y votacion

1. Betzabet reporta un post de Aaron.
2. Guillermo vota DELETE.
3. Aaron vota DELETE.
4. Un cuarto admin vota KEEP.
5. Con 3 votos (2–1) el sistema resuelve con `RESOLVED_DELETE`.
6. El post se elimina automaticamente.

11.4.3. Moderacion directa

- El dueno del canal puede borrar publicaciones ajenas.
- El borrado se registra como `Moderacion del dueno del canal`.
- Un ADMIN puede borrar en cualquier canal.
- Un STUDENT solo puede borrar sus propios posts.

11.5. Resultado

Estado al cierre

- Sistema de moderacion completo con votacion colegiada.
- Jerarquia de roles con reglas bien definidas.
- Auditoria de todas las acciones administrativas.
- Base solida para la interfaz de administracion en el cliente Flutter.

Capítulo 12

Flutter: reportar, moderar y administrar

12.1. Objetivo

Implementar en el cliente Flutter toda la interfaz administrativa y de moderacion:

- Boton de reportar en publicaciones y canales.
- Menu contextual en cada publicacion con Reportar y Eliminar.
- Pantalla de administracion con tres pestanas: Reportes, Usuarios y Auditoria.
- Detalle del reporte con votacion.
- Gestion de roles y baneo de usuarios.
- Consulta del log de auditoria.

12.2. Contexto

El backend de la Fase 11 ya exponia todos los endpoints. El cliente no tenia ninguna interfaz que los consumiera. Se dividio el trabajo en cinco sub-fases: D2 (boton de reportar), D3 (lista de reportes), D4 (panel completo), D5 (moderacion directa).

12.3. Desarrollo

12.3.1. Modelos nuevos

- `Report`: mapea la respuesta de `/api/admin/reports`.
- `AdminUser`: mapea `/api/admin/users`.
- `AuditLog`: mapea `/api/admin/audit`.

12.3.2. Providers nuevos

- `ReportProvider`: crear reportes desde cualquier usuario.
- `AdminProvider`: carga de reportes, usuarios y auditoria, ademas de votar, cambiar rol y banear.
- `PostProvider.delete()`: borra publicaciones y actualiza el estado local.

12.3.3. Wigets reutilizables

`ReportDialog`

Modal reutilizable con:

- Radio buttons con las cinco razones (`SPAM`, `INAPPROPRIATE`, `MISINFORMATION`, `HARASSMENT`, `OTHER`).
- Campo de detalles opcional.
- Validacion y feedback con SnackBar.

Se expone como metodo estatico:

```
static Future<void> show(BuildContext context, {
  required String targetType,   // 'POST' | 'CHANNEL'
  required String targetId,
  required String targetLabel,
});
```

12.3.4. Menu contextual en publicaciones

Se reemplazo el icono de reportar por un `PopupMenuButton` que muestra las opciones segun los permisos del usuario actual.

```
final isOwn = currentUserId != null && post.authorId == currentUserId;
final canDelete = isOwn || canModerateAny;
final canReport = !isOwn;

PopupMenuButton<String>(
  onSelected: (value) {
    if (value == 'report') ReportDialog.show(...);
    else if (value == 'delete') onDelete();
  },
  itemBuilder: (_) => [
    if (canReport) PopupMenuItem(value: 'report', child: ...),
    if (canDelete) PopupMenuItem(value: 'delete', child: ...),
  ],
);
```

12.3.5. Pantalla `AdminScreen`

Contiene un `TabBar` con tres sub-pestanas:

- `AdminReportsTab`: lista con filtros por estado.
- `AdminUsersTab`: listado con buscador y acciones.
- `AdminAuditTab`: bitacora con iconos por tipo de accion.

Solo es visible para ADMIN y SUPERADMIN (verificado con `auth.isAdminOrSuper`).

12.3.6. `AdminReportsTab`

Filtros por estado:

```
static const _filters = [
  ('PENDING', 'Pendientes'),
  ('RESOLVED_KEEP', 'Conservados'),
  ('RESOLVED_DELETE', 'Eliminados'),
  ('ALL', 'Todos'),
];
```

Cada tarjeta muestra:

- Chip de tipo (Post/Canal).
- Chip de razon.
- Chip de estado con color.
- Preview del contenido reportado.
- Contador de votos `0/3`, `1/3`, etc.
- Indicador de voto propio si el usuario ya voto.

12.3.7. `ReportDetailScreen`

Pantalla de detalle con:

- Contenido reportado y su ubicacion.
- Motivo y detalles del reporte.
- Informacion del reporter.
- Contador de votos con barra de progreso.
- Botones de voto (Conservar / Eliminar).
- Estado de resuelto si aplica.

Antes de votar, se pide un comentario opcional en un dialogo.

12.3.8. `AdminUsersTab`

Lista con:

- Avatar con inicial.
- Nombre, email y badge de rol con color.
- Badge “Tu” en el usuario actual.
- Badge “BANEADO” si esta deshabilitado.
- Boton de tres puntos que abre un bottom sheet con las acciones.

12.3.9. Bottom sheet de acciones

Para ADMIN muestra solo promociones (Estudiante, Profesor, Administrador). Para SUPERADMIN agrega:

- Opcion de nombrar SUPERADMIN.
- Boton de Banear o Reactivar.

Cada accion pide confirmacion en un dialogo antes de ejecutarse.

12.3.10. `AdminAuditTab`

Muestra el log con:

- Icono y color segun la accion (`VOTE_KEEP`, `VOTE_DELETE`, `CHANGE_ROLE`, `BAN_USER`, `UNBAN_USER`, `DELETE_POST`).
- Nombre del admin.
- Detalles.
- Fecha y hora.

Un banner superior indica si esta viendo todo el log (SUPERADMIN) o solo el propio (ADMIN).

12.3.11. Cambios a archivos previos

- `home_shell.dart`: se agrego la pestana Admin condicional segun el rol.
- `channel_detail_screen.dart`: se agrego el menu contextual por publicacion.
- `post_provider.dart`: se agrego el metodo `delete`.
- `main.dart`: se registraron `ReportProvider` y `AdminProvider`.

12.4. Verificacion

- Un estudiante puede reportar publicaciones y canales.
- Un administrador ve la pestana Admin.
- El SUPERADMIN ve tres sub-pestanas y puede votar, gestionar usuarios y consultar el log completo.
- Un ADMIN solo ve sus propias acciones en el log.
- El dueno del canal puede borrar publicaciones ajenas sin votacion.

12.5. Resultado

Estado al cierre

- Interfaz completa de reportes y moderacion.
- Gestion de usuarios con jerarquia de roles.
- Auditoria visible segun el rol.
- Moderacion directa por dueno del canal.

Capítulo 13

Distintivo visual de canales institucionales

13.1. Objetivo

Marcar visualmente los canales segun el rol de su creador para que los usuarios distingan rapidamente entre:

- Canales creados por profesores (contenido academico).
- Canales creados por administradores (contenido oficial).
- Canales creados por estudiantes (sin distintivo).

13.2. Contexto

Una vez integrado el sistema de roles, era natural que la interfaz reflejara esa jerarquia en los canales. Un canal creado por un profesor tiene un peso academico distinto al de un canal creado por un estudiante, y un canal oficial debe ser reconocible a primera vista.

13.3. Desarrollo

13.3.1. Cambio en el backend

Se agrego el campo `ownerRole` a los dos DTOs de canal:

```
public record ChannelSummaryResponse(
        UUID id,
        String name,
        String category,
        boolean publicChannel,
        String ownerDisplayName,
        String ownerRole,            // NUEVO
        long subscriberCount,
        Instant createdAt
) {}
```

Y el mismo campo en `ChannelResponse`. En `ChannelService` se actualizaron los dos metodos privados que construyen la respuesta.

13.3.2. Modelo en Flutter

```dart
class Channel {
  final String ownerRole;
  // ...

  bool get isOfficial => ownerRole == 'ADMIN' || ownerRole == '
      SUPERADMIN';
  bool get isProfessorOwned => ownerRole == 'PROFESSOR';
}
```

13.3.3. Wiget `ChannelOwnerBadge`

```dart
class ChannelOwnerBadge extends StatelessWidget {
  final String ownerRole;
  final bool compact;

  @override
  Widget build(BuildContext context) {
    if (ownerRole == 'STUDENT') return const SizedBox.shrink();

    if (ownerRole == 'PROFESSOR') {
      return _badge(
        icon: Icons.school_outlined,
        label: 'Profesor',
        color: NotiuamColors.info,
      );
    }

    if (ownerRole == 'ADMIN' || ownerRole == 'SUPERADMIN') {
      return _badge(
        icon: Icons.verified_outlined,
        label: 'Oficial',
        color: NotiuamColors.orangeDark,
      );
    }

    return const SizedBox.shrink();
  }
}
```

13.3.4. Ubicaciones del badge

El badge se muestra en:

- `ExploreScreen`: junto al nombre de cada canal en la lista.
- `ChannelDetailScreen`: debajo del nombre del canal.
- `_ChannelList` en `home_shell`: junto al nombre en “Mis canales” y “Suscritos”.

En todas se usa una version compacta cuando el espacio es reducido.

13.4. Verificacion

- Canales creados por PROFESSOR muestran badge azul con etiqueta “Profesor”.
- Canales creados por ADMIN o SUPERADMIN muestran badge naranja con etiqueta “Oficial”.
- Canales creados por estudiantes no muestran badge.

13.5. Resultado

Estado al cierre

- Distincion visual clara segun el rol del creador.
- Coherencia con la jerarquia de roles definida en el backend.
- Refuerzo de la confianza en canales oficiales y academicos.